PIPELINE DRY RUN · APPLE M4 PRO · 8 NOV 2026

Kolibri W4A16 pipeline on a MacBook

The FP8 → BF16 → GPTQ 4-bit → evaluation pipeline was run end to end twice on a 24 GB MacBook: once on the tiny random Kolibri-1 test model, and once on a real block-FP8 model, Qwen3-0.6B-FP8. This page reports memory and CPU use for every step, and how close the 4-bit Qwen3 model stays to the original.

`M4 Pro · 12 cores · 24 GB` `torch 2.14.1` `transformers 5.21.0` `llm-compressor 0.15.0` `compressed-tensors 0.20.1`

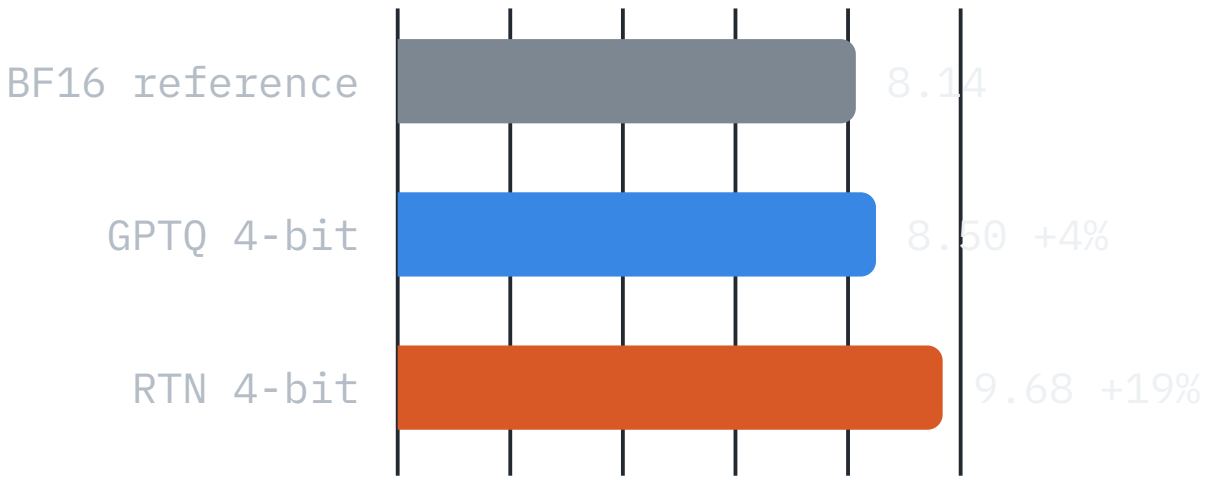
Perplexity increase, GPTQ	Top-1 agreement with BF16	Peak process memory	Swap used
+4.4%	**84.8%**	**11.2 GB**	**0 MB**
README target: under 3–5%	RTN: 75.8%	quality eval, 3 models loaded	in every step

OUTPUT QUALITY · QWEN3-0.6B

GPTQ keeps the 4-bit model close to the original

Measured on 64 held-out chat samples (19,368 tokens; 70% English from open-perfectblend, 30% German from Aya), none of which were in the 128-sample calibration set. GPTQ is compared with plain round-to-nearest (RTN) 4-bit, using the same W4A16 group-128 scheme with no calibration. All models ran on the CPU.

Perplexity on held-out data — lower is better



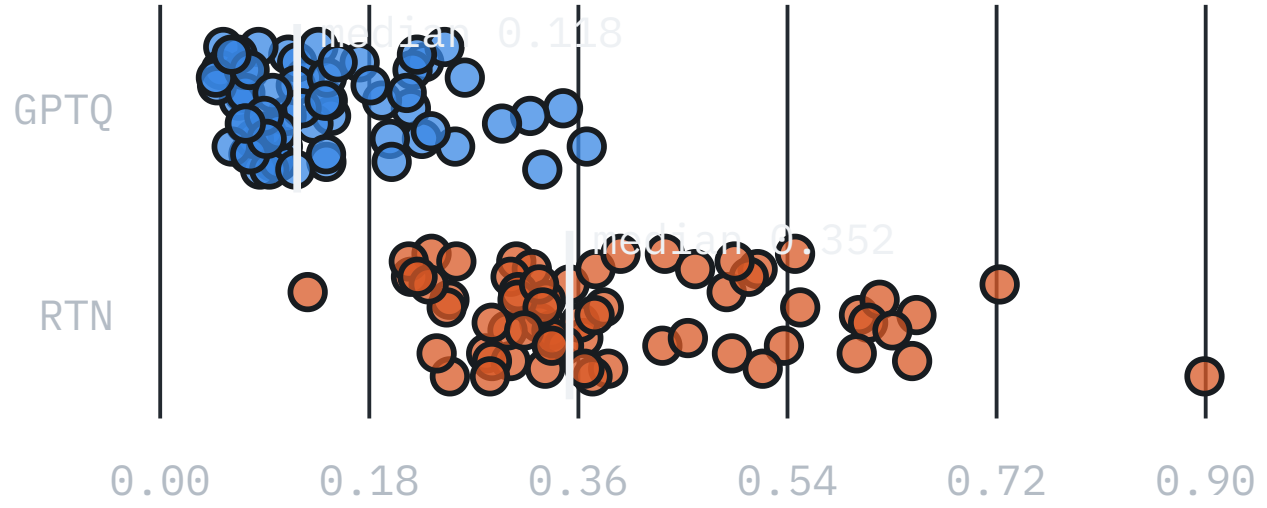
0 2 4 6 8 10

MODEL	SIZE	PERPLEXITY	VS BF16	KL VS BF16	TOP-1 AGREEMENT
BF16 reference	1,434 MB	8.138	—	—	—
GPTQ 4-bit	514 MB	8.496	+4.4%	0.132	84.8%
RTN 4-bit	514 MB	9.681	+19.0%	0.357	75.8%

KL divergence per held-out sample

GPTQ RTN

Each dot is one sample's mean per-token KL from the BF16 model; the tick is the median. Lower means the output distribution drifted less.



Sample generations

Greedy decoding, 90 new tokens, thinking turned off. A 0.6B model is unreliable on facts even at full precision, so compare the three columns with each other rather than judging the answers.

Erkläre kurz, was Quantisierung bei LLMs bedeutet.

BF16 reference

Quantisierung ist die Prozessierung von kontinuierlichen Daten in discrete Werte, um die LLMs zu optimieren. Dies reduziert die Komplexität und ermöglicht eine effizientere Auswertung.

GPTQ 4-bit

Quantisierung ist ein Prozess, bei dem LLMs (Large Language Models) ihre Modelle in eine strukturierte, quantisierte Form überschrieben werden. Dies ermöglicht die Erstellung von Modelle mit spezifischen Lernalgorithmen, die in der Praxis oft auf die Komplexität der Lernalgorithmen in der Praxis reduziert werden. Quantisierung ist eine technolog

RTN 4-bit

Quantisierung bei LLMs bedeutet, dass die LLMs ihre Erkenntnisse in Werte oder Werte-Gruppen (z. B. Intervalle) zerlegten, um die Erkenntnisse in eine binäre Form zu bringen. So kann die LLM die Erkenntnisse in einer binären Form darstellen, die dann in einer Schicht oder einer Schichtenstruktur organ

All three answer in German. Wording differs but none is clearly broken.

Explain in two sentences why the sky is blue.

BF16 reference

The sky appears blue because of the way light interacts with the Earth's atmosphere, scattering shorter wavelengths of light (blue) while longer wavelengths (red and orange) are scattered more, resulting in a perception of blue.

GPTQ 4-bit

The sky appears blue because it contains a mixture of nitrogen and oxygen molecules, which absorb red light and emit blue light, creating a colorless and blue sky.

RTN 4-bit

The sky appears blue because it contains a mixture of molecules and particles that scatter and reflect light, creating the illusion of a blue color. This phenomenon is a result of the way light interacts with the atmosphere and the Earth's surface.

None is fully right. The reference says red light scatters more, which is backwards. GPTQ invents absorption and emission. RTN is vague but not wrong.

Write a Python function that returns the n-th Fibonacci number.

BF16 reference

Here's a simple Python function that returns the **n-th Fibonacci number**:

```python
def fibonacci(n):
    if n <= 0:
        return 0  # Handle n=0 or negative numbers
    elif n == 1:
        return 1
    else:
        a, b = 0, 1
        for _ in range(2, n + 1):
            a, b = b,
```

GPTQ 4-bit

Here's a simple Python function that returns the **n-th Fibonacci number**:

```python
```

```
python
def fibonacci(n):
    if n <= 0:
        return 0  # or handle edge cases as needed
    elif n == 1 or n == 2:
        return 1
    else:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a +
```

RTN 4-bit

Here's a simple and efficient way to write a function that returns the n-th Fibonacci number in Python:

```python
def fibonacci(n):
    if n <= 0:
        return 0
    elif n == 1 or n == 2:
        return n
    else:
        a, b = 0, 1
        for _ in range(1, n+1):
            a, b = b, a
```

All three write the same loop structure. Each has small edge-case differences.

RESOURCE USAGE

Every step fit in memory with room to spare

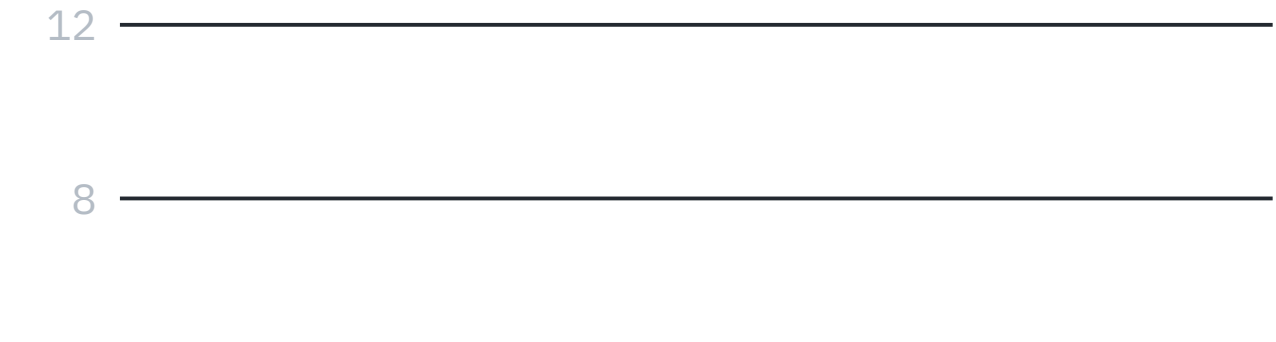
Each step ran under a sampler that recorded the memory and CPU of the process and its children every 0.5 s, alongside a live dashboard of the whole machine. About 12 GB was already in use by other apps before the runs.

Process memory over time, Qwen3 steps

GB resident · hover for detail

02 GPTQ calibration

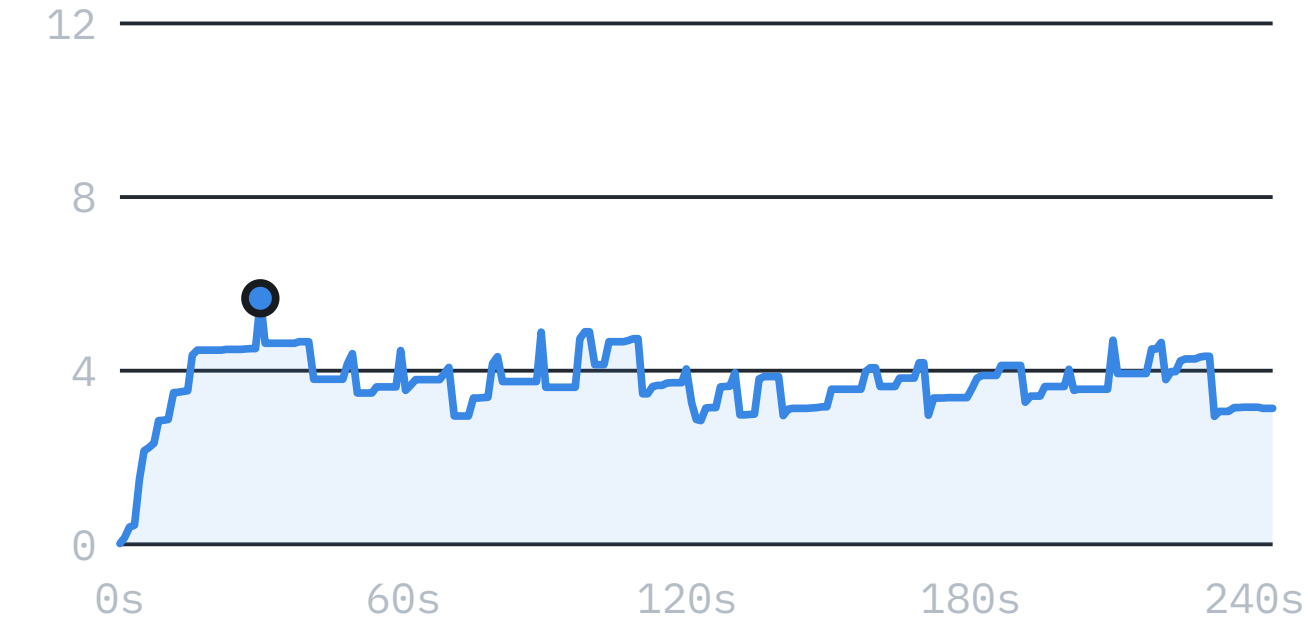
`damping 0.05 · peak 1.3 GB · 49 s`





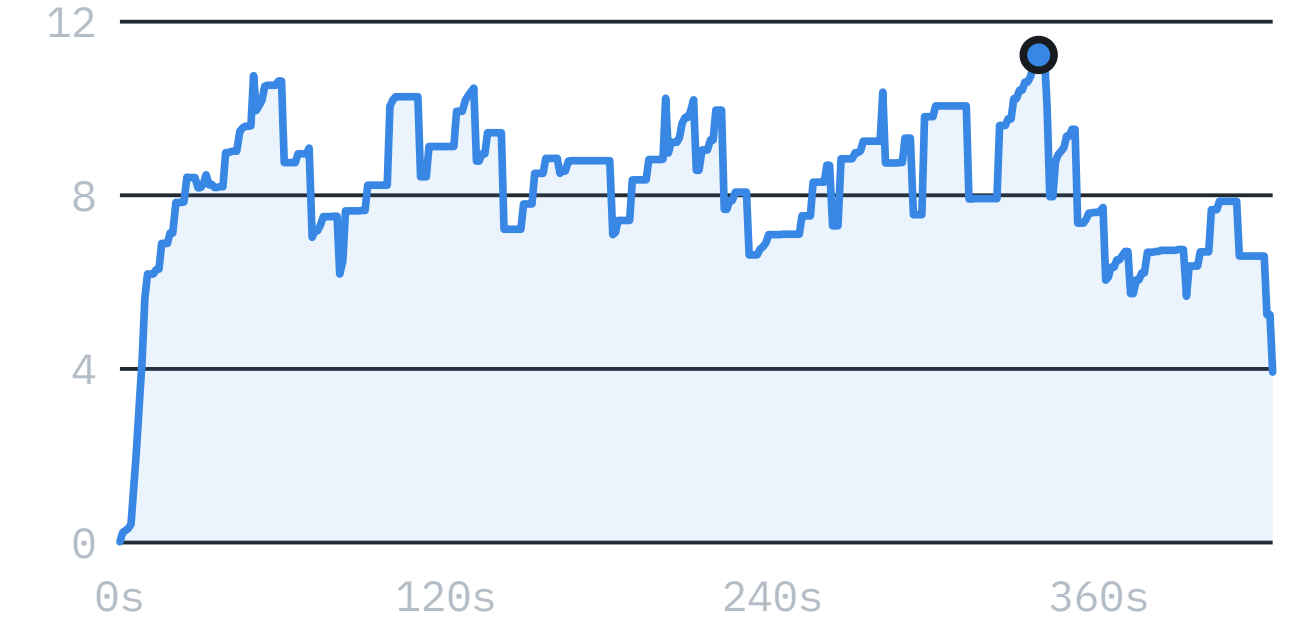
03 perplexity compare

2 models, one at a time · peak 5.7 GB · 244 s



Quality eval

3 models at once + RTN build · peak 11.2 GB · 424 s



STEP	WALL TIME	PEAK MEMORY	AVG CPU	PEAK CPU	SWAP
TINY RANDOM KOLIBRI-1					
Build test model	5.1 s	453 MB	91%	98%	0 MB

01 FP8 → BF16	1.5 s	212 MB	100%	100%	0 MB	p
02 GPTQ (275 linears)	18.4 s	656 MB	49%	100%	0 MB	p
03 compare	6.1 s	422 MB	150%	281%	0 MB	p
QWEN3-0.6B-FP8						
01 FP8 → BF16 (1.5 GB out)	1.5 s	865 MB	156%	172%	0 MB	p
02 GPTQ, damping 0.01	12.8 s	1.27 GB	58%	100%	0 MB	c
02 GPTQ, damping 0.05 (196 linears)	48.5 s	1.25 GB	42%	100%	0 MB	p
03 compare (64 samples)	244.3 s	5.67 GB	721%	798%	0 MB	p
Quality eval: KL, top-1, RTN	424.2 s	11.23 GB	698%	977%	0 MB	p

CPU is a percentage of one core, so 800% means about eight of the 12 cores were busy. GPTQ itself ran on the Mac GPU (MPS) and used under 2% of its 19.1 GB, which the process memory figures don't show.

FINDINGS

What broke and what was changed

Fixed

GPTQ crashed on Qwen3 with the default damping

At `dampening_frac=0.01`, calibration stopped on `model.layers.2.mlp.down_proj` with `linalg.cholesky: … not positive-definite`. The failure is in the factorization of the inverted Hessian, which llm-compressor's round-to-nearest fallback doesn't cover. Damping 0.05 fixed it with no layers falling back. The script now has `--dampening-frac` (default unchanged at 0.01) and the README explains when to raise it.

Mac only

Loading a model onto the Mac GPU crashes step 03

`03_compare_ppl.py` loads with `device_map="auto"`, which picks the Mac GPU (MPS). The process then dies silently with exit code –5. Loading on the CPU works, so step 03 ran through a small wrapper that forces CPU. On the intended NVIDIA machine this doesn't arise, and the script was left unchanged.

Passed

Tiny Kolibri-1 model, all steps

FP8 dequantization reproduced all 318 tensor names with about 2.6% relative error, which is FP8 rounding. GPTQ quantized all 275 linear layers including every expert and saved `pack-quantized` W4 group-128 tensors, and the result reloads and generates. Its perplexity numbers mean nothing because the weights are random.

> Note **Dataset streaming left a process behind**
>
> The script that streamed calibration data wrote both files and then hung on exit, holding 2.4 GB. The live monitor showed it and it was stopped. This came from the test setup, not the pipeline.

SCOPE

What this test does and doesn't show

- Qwen3-0.6B is dense. It exercises the real FP8 format, dequantization, GPTQ and the compressed-tensors output, but not Kolibri's 384-expert MoE path. That path was only checked on the tiny random model.
- Small models lose more at 4 bits than large ones, so +4.4% perplexity at 0.6B is a pessimistic stand-in for 78B.
- Peak memory scales with model size. The full Kolibri-1 needs about 156 GB for its BF16 weights and an NVIDIA GPU with about 30 GB free, so it can't run on this Mac. The README's rented H100 with 256 GB of RAM is still the plan.
- The full-size run could hit the same Cholesky failure. If it does, add `--dampening-frac 0.05` to step 2 in `run_all.sh`.

Model	Qwen/Qwen3-0.6B-FP8 (1.0 GB, block FP8 128×128, `weight_scale_inv`)
Calibration	128 chat samples, max 1,024 tokens (119 after dropping samples under 64 tokens), batch size 4
Scheme	W4A16, symmetric INT4, group size 128, `actorder=static`, `lm_head` kept in BF16, damping 0.05
Evaluation	64 held-out samples, max 1,024 tokens, CPU, BF16 dequantized model as the reference